# Logging

chukmunnlee@nus.edu.sg

© Copyright National University of Singapore

# What is Logging ?

- Essential part of application and operating system
  - Involve the recording of information about events that occurs
- Logs can be used for
  - Security
    - Perform monitoring, detecting and analyzing events
  - Auditing
    - Compliance
  - Observability
    - Troubleshoot
- Logging examples
  - Audit log, transaction log, user activity log
- Sources of logs
  - Application, operating system, databases, firewall

© Copyright National University of Singapore

# What Information are Logged?

- Events eg. system startup and shutdown
- System health eg. cluster heartbeat
- Threshold reaching dangerous level or exceeded eg. memory utilization is at 80%, disk is almost full
- Authentication and authorization eg. user fred is logged in as root
- System configuration changes eg. status of operating system component have been upgraded
- Application access and request eg. HTTP request details and status

© Copyright National University of Singapore

# Common Logging Format - 1

- logfmt - structured logging format that uses space-separated key/value pairs

```
level=info ts=2025-09-08T10:15:23.812Z msg="Started nightly job"
job=reindex_search host=app-3 duration_ms=152
```

- JSON - logs are formatted using JSON

```
{ "ts": "2025-09-08T10:15:23.812Z", "level": "info",  "msg": "Started nightly
job", "job": "reindex_search", "host": "app-3", "duration_ms": 152 }
```

- Syslog - standard log format used by operating systems like Linux

```
<134>Sep  8 10:15:23 app-3 myapp[12345]: Started nightly job
'reindex_search' duration_ms=152
```

© Copyright National University of Singapore

# Common Logging Format - 2

- Common Log Format (CLF) - standard text based format for web server access

```
203.0.113.42 - - [08/Sep/2025:10:15:23 +0000] "GET /health HTTP/1.1" 200 2
```

- Extended CLF - customizable log format

```
#Version: 1.0
#Date: 2025-09-08 10:15:23
#Fields: date time c-ip cs-method cs-uri-stem sc-status time-taken
2025-09-08 10:15:23 203.0.113.42 GET /search 200 53
```

© Copyright National University of Singapore

# Trace Record Structure

- Share the same common structure as tracing
  - resource, instrumentation and attributes
- Fields include
  - timestamp,
  - trace id, span id - used for correlating the log to a trace
  - severity text, severity number - log level eg INFO, DEBUG
  - body - the log itself
  - resource - source of the log, contains service.name, service.version, etc
  - attributes - additional context information, key/value pair
- These fields can be accessed and manipulated by OTel pipelines

© Copyright National University of Singapore

# Example - OTLP Log Record

```
otel-1       | 2026-01-11T23:33:50.557Z  info    Logs    {"resource": {"service.instance.id": "61abe7f4-28e4-4ef1-b38e-39d3d96c6f0d", "service.name": "otelcol-contrib", "service.ver
sion": "0.142.0"}, "otelcol.component.id": "debug", "otelcol.component.kind": "exporter", "otelcol.signal": "logs", "resource logs": 1, "log records": 11}
otel-1       | 2026-01-11T23:33:50.587Z  info    ResourceLog #0
otel-1       | Resource SchemaURL:
otel-1       | Resource attributes:
otel-1       |      -> host.name: Str(batcomputer)
otel-1       |      -> host.arch: Str(amd64)
otel-1       |      -> host.id: Str(b93676df0cfa42289ae466b615b58c1b)
otel-1       |      -> process.pid: Int(1589364)
otel-1       |      -> process.executable.name: Str(node)
otel-1       |      -> process.executable.path: Str(/usr/bin/node)
otel-1       |      -> process.command_args: Slice(["/usr/bin/node","--require","./src/instrument.js","/home/cmlee/src/soda/cno/work/day02/solution/books/src/main.js"])
otel-1       |      -> process.runtime.version: Str(24.0.1)
otel-1       |      -> process.runtime.name: Str(nodejs)
otel-1       |      -> process.runtime.description: Str(Node.js)
otel-1       |      -> process.command: Str(/home/cmlee/src/soda/cno/work/day02/solution/books/src/main.js)
otel-1       |      -> process.owner: Str(cmlee)
otel-1       |      -> service.name: Str(books)
otel-1       |      -> service.version: Str(1.0.0)
otel-1       | ScopeLogs #0
otel-1       | ScopeLogs SchemaURL:
otel-1       | InstrumentationScope books 1.0.0
otel-1       | LogRecord #0
otel-1       | ObservedTimestamp: 2026-01-11 23:33:45.445 +0000 UTC
otel-1       | Timestamp: 2026-01-11 23:33:45.445 +0000 UTC
otel-1       | SeverityText: INFO
otel-1       | SeverityNumber: Info(9)
otel-1       | Body: Str(getBookByISBN13: isbn13=9789042003408)
otel-1       | Attributes:
otel-1       |      -> service.name: Str(books)
otel-1       |      -> service.version: Str(1.0.0)
otel-1       |      -> db.collection.name: Str(books)
otel-1       |      -> trace_id: Str(c082e8b0d3121136e31669e1ce0dc7fc)
otel-1       |      -> span_id: Str(98a02114c85c935a)
otel-1       |      -> trace_flags: Str(01)
otel-1       |      -> timestamp: Str(2026-01-11T23:33:45.445Z)
otel-1       | Trace ID: c082e8b0d3121136e31669e1ce0dc7fc
otel-1       | Span ID: 98a02114c85c935a
otel-1       | Flags: 1
otel-1       | LogRecord #1
otel-1       | ObservedTimestamp: 2026-01-11 23:33:45.445 +0000 UTC
otel-1       | Timestamp: 2026-01-11 23:33:45.445 +0000 UTC
otel-1       | SeverityText: ERROR
otel-1       | SeverityNumber: Error(17)
otel-1       | Body: Str(Database error injection triggered in getBookByISBN13: ER_TOO_MANY_USER_CONNECTIONS: Too many connections)
otel-1       | Attributes:
```

© Copyright National University of Singapore

# Logging Architecture

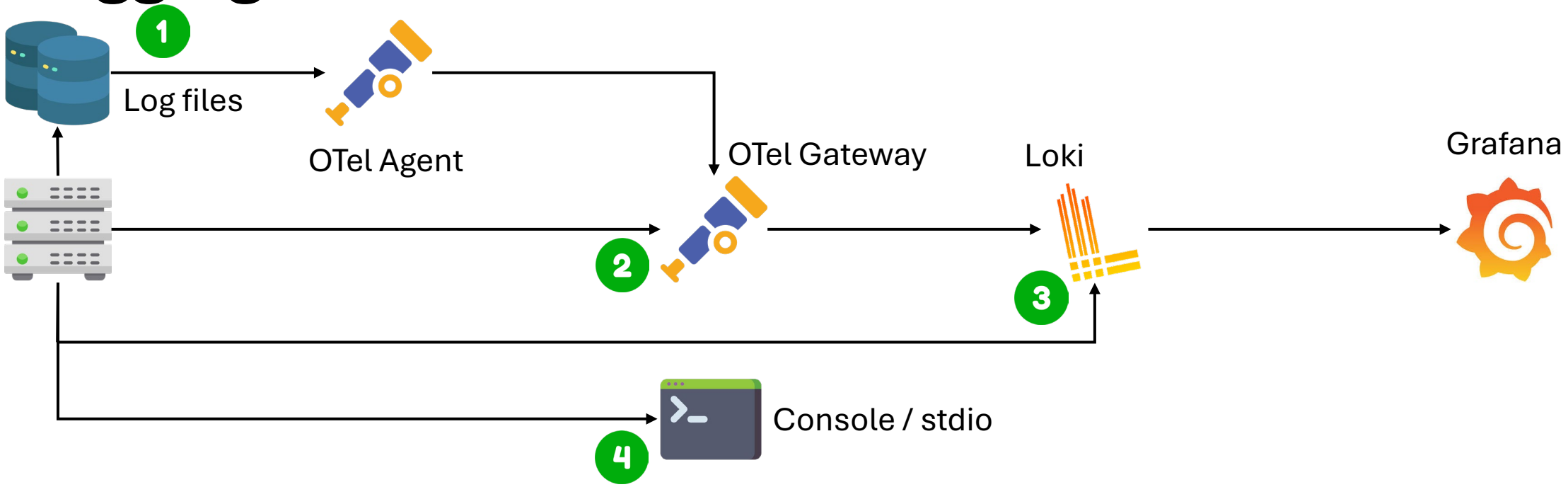
© Copyright National University of Singapore

# Local and Central Logging

- Types of logging
  - Central logging stream logs from application, operating system, infrastructure services to a centralized location
  - Local logging is where logs are written on the local systems first
    - May be processed locally or may be shipped by a logging agent to a centralized location
- Common architectures
  - Cloud provider like AWS, GCP will have their framework for managing and shipping logs
  - ELK - Elastic Search, Logstash/Fluentd and Kibana
  - LGTM - Loki, Grafana, Tempo and Mimir/Prometheus

© Copyright National University of Singapore

# Logging Architecture




1. • Application write logs to files on the server. OTel agent running locally scrapes the logs
   - Agent forwards the logs to OTel gateway which is then shipped to Loki
2. • Application write logs directly to gateway
3. • Application write logs directly to Loki
4. • Application write logs to console/stdout
   - No way of capturing logs unless redirected to a file

© Copyright National University of Singapore

# Loki

- Open-source multi-tenant log aggregator
  - Centralizes and consolidates logs from different sources
  - Core component of the Grafana stack
- Major features of log aggregators
  - Parsing logs, structured and unstructured into key/value pair for easy querying
  - Enriching logs by adding additional context like IP address, service names, global time, etc.
  - Redacting sensitive data from logs such as personal information, passwords, etc.
- Logs are queried with LogQL
- Does not have a UI, rely on external tool like Grafana

© Copyright National University of Singapore

# Configure Loki Data Source in Grafana

Loki

Type
Loki

Alerting
Supported

Explore data | Build a dashboard

Type: Loki

Settings

Name: Loki

Default

Before you can use the Loki data source, you must configure it below or in the config file. For detailed instructions, view the documentation.

## Connection

URL *: http://loki:3100

## Authentication

Authentication methods

Choose an authentication method to access the data source

Authentication method

No Authentication

© Copyright National University of Singapore

# Application Logging

© Copyright National University of Singapore

# Application Logging

- Logging is a mature field in software development
  - Lots of popular libraries
  - Eg. SLF4J, Morgan, Loguru, Zap
- OpenTelemetry works with existing logging libraries
  - Well established, developers are generally now willing to change their code base because of logging
  - Leverages these libraries to write logs either to disk or send to central logging system

© Copyright National University of Singapore

# Application Log Files

- Two broad classification of log files
  - Enriched by auto instrumentation
    - Will contain injected information like **`trace_id`** and **`span_id`** as part of the log line
  - Native log format, not auto instrumentation
    - Eg. syslog, Nginx
- Logging involves existing logging format to OTel record format
  - Involves manipulating the log lines either at the agent or at the gateway collector to suit the backend tools with varied requirements
  - Unlike traces and metrics where backend tools vendor support is more uniform

© Copyright National University of Singapore

# Winston Logger

- Popular JavaScript logging library
  - https://github.com/winstonjs/winston
- Modular design based on the following core concepts
  - Logger - the main object for writing logs
    - Supports different logs levels
    - See levels - https://github.com/winstonjs/winston?tab=readme-ov-file#logging
  - Transport - storage and transport mechanism
    - Supports file, console and HTTP out of the box
    - Multiple transport support viz. write to multiple outputs
  - Formats - logging data structure
    - Supports JSON, logfmt, logstash, unstructured text or construct your custom format
    - Beautify logs with colorized

© Copyright National University of Singapore

# Example - Create a Logger

```
const logger = winston.createLogger({
  level: 'debug',
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({
      filename: '/var/logs/app.log',
      level: 'info',
      handleExceptions: true,
      handleRejections: true
    })
  ],
  format: json(),
  defaultMeta: {
    [ ATTR_SERVICE_NAME ]: metadata.name,
    [ ATTR_SERVICE_VERSION ]: metadata.version
  }
})
```

Factory method to create a logger

Only log if log is at debug or less than this level

Write to console

Where to shipped the logs to

Save to file

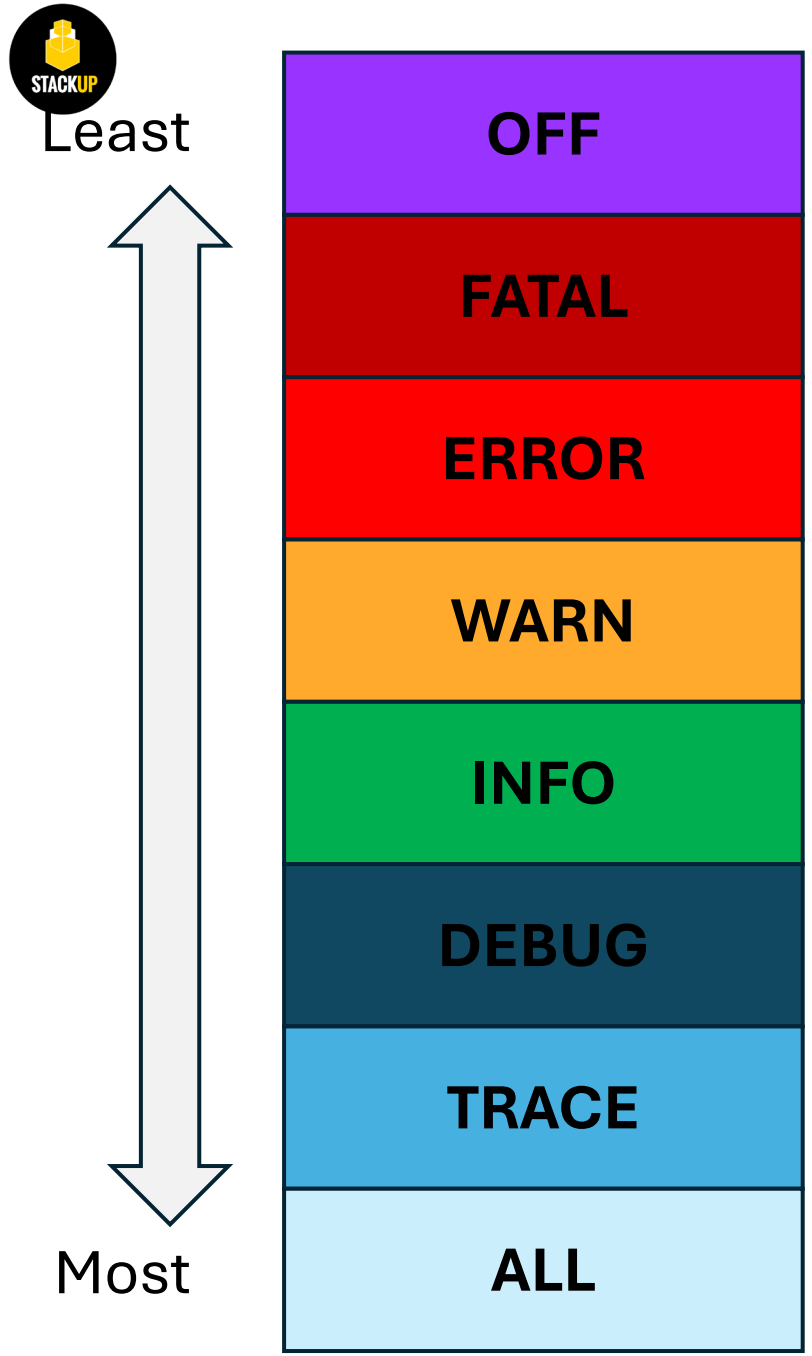
Write to log file if log level is the same as or lower than this level

Log any unhandled exceptions and promise rejections

Write the log in JSON format

Additional metadata to be included in the log line

© Copyright National University of Singapore




# Logging Levels

- To categorize and prioritize the log messages
- Common logging levels
  - Debug - fine-grained events that are useful for debugging applications
  - Information - messages that highlight the progress of the application
  - Warning - potentially harmful situation
  - Error - error that might still allow the application to continue
  - Fatal - error that impact the integrity of the application

© Copyright National University of Singapore

# Logging and Log Level with Winston

```
logger.error('This is an error message')
logger.warn('This is a warning message')
logger.info('This is an info message')
logger.http('This is a http request message')
logger.verbose('This is a verbose message')
logger.debug('This is a debug message')
```

- The purpose of logging level is having control over how much logs are produced
  - Eg. more logs in development but only warnings and errors in production
- Levels allows operation teams to quickly filter the logs
- Set the logging level

```
LOG_LEVEL=warn node --require ./instrument.js main.js
```

© Copyright National University of Singapore

# Example - JSON Log Line

Log level

Actual log message

```
{
  "level": "info",
  "message": "This is an info message",
  "service.name": "books",
  "service.version": "1.0.0",
  "span_id": "a79f5cecccb1e595",
  "trace_flags": "01",
  "trace_id": "af375188e88d85165e9f04326ec587b1"
}
```

Metadata from **defaultMeta**

Labels injected by auto instrumentation
Helps to correlate traces to logs

© Copyright National University of Singapore

# Workshop 2
# Task 1

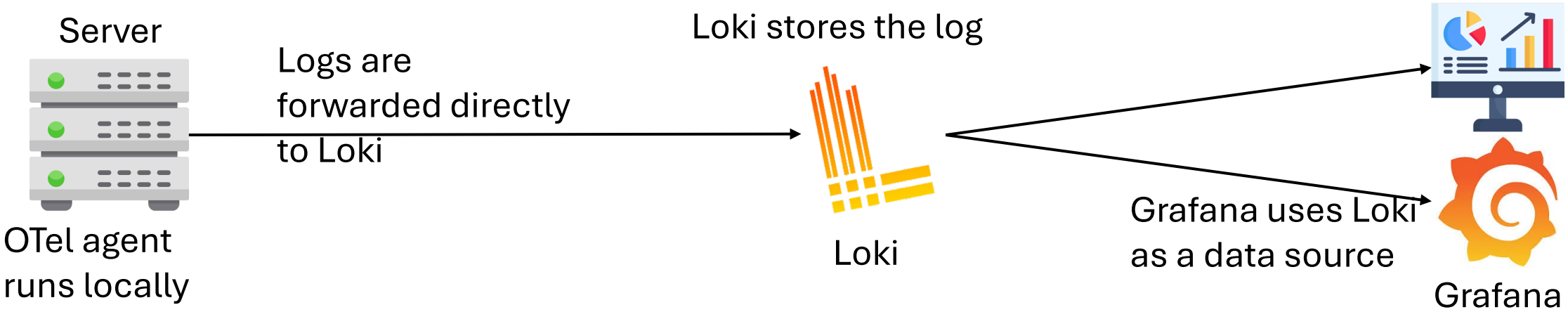
© Copyright National University of Singapore

# Forwarding Logs

Direct to Log Aggregator from Application

© Copyright National University of Singapore

# Forwarding Logs Directly to Loki




- Simple to setup, not much resources is required
- Logs are in native Loki format, no pre-processing required
- Applications are committed to Loki for log aggregation

© Copyright National University of Singapore

# Using `LokiTransport`

- Winston has a flexible architecture for storing and forwarding logs
  - Configure plugins to forward logs to their destination
  - Eg. to file, console, MongoDB, syslog, etc
- `LokiTransport` forwards logs to Loki endpoints
  - From package winston-loki

```
const lokiTransport = new LokiTransport({
  host: 'http://loki:3100',
  ... // other configuration
})

const logger = winston.createLogger({
  ...
  transports: [
    new winston.transports.Console(),
    lokiTransport
  ]
})
```

© Copyright National University of Singapore

# Loki Configuration

```
loki -config.file=loki-config.yaml
```

```yaml
server:
  http_listen_port: 3100
  grpc_listen_port: 9060

limits_config:
  metric_aggregation_enabled: true
  allowed_structured_metadata: true

pattern_ingester:
  enabled: true
  metric_aggregation:
    loki_address: localhost:3100
```

Enable logs to be aggregated with LogQL into to time-series metrics

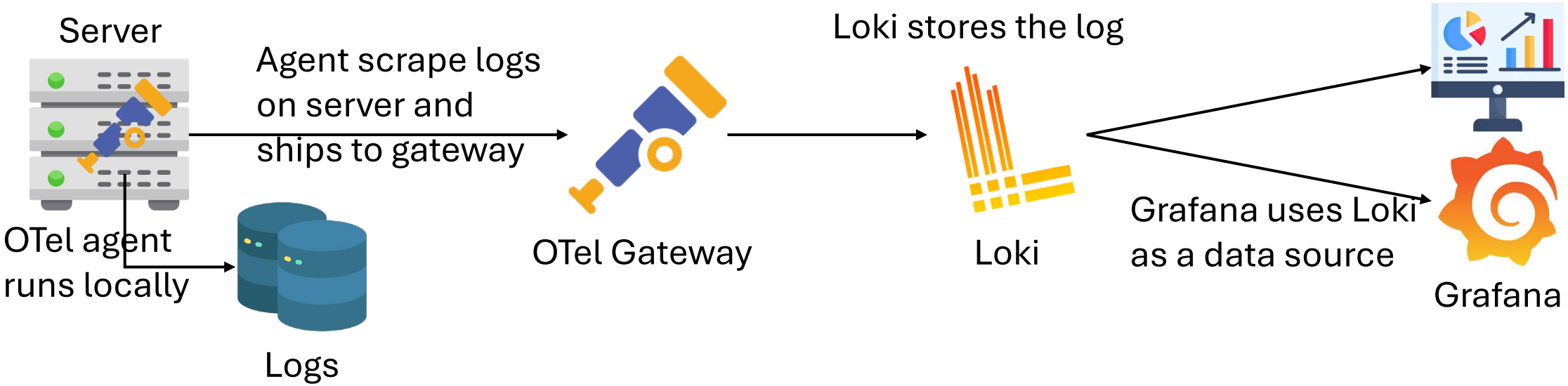
© Copyright National University of Singapore

# Forwarding Logs

With Local Agent

© Copyright National University of Singapore

# Forwarding Local Logs




- Log files can be stored locally on the server
  - Operating system logs in **`/var/log`** directory on Linux
  - Applications (legacy?) that saves their logs locally on the server
- Logs need to be shipped to log aggregator by an agent running locally
  - Eg. OTel collector, Promtail, Vector, Fluentbit
- Manual and more complex
  - Need to populate OTel log records with the correct information scrapped from the log files

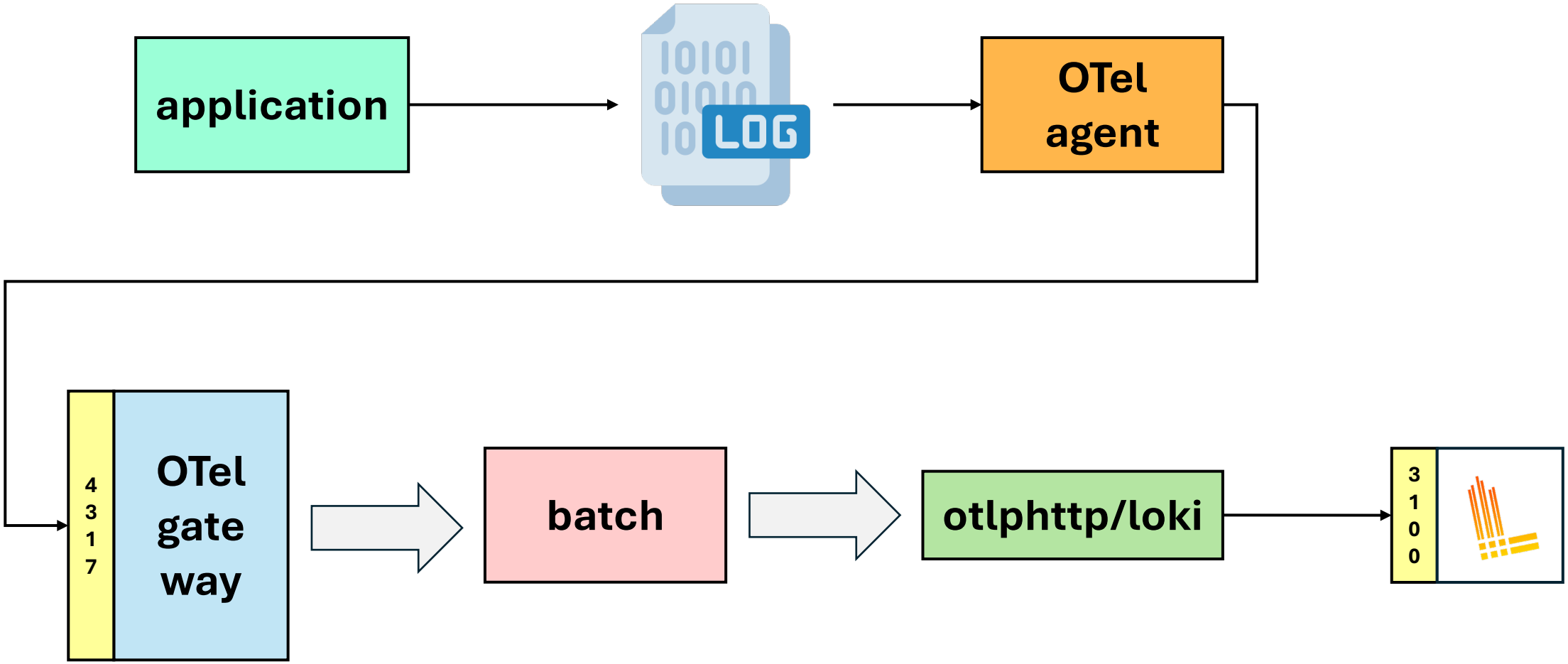
© Copyright National University of Singapore

# Agent Log Scraping

- Primary goal of agents is to convert log lines to OTel log format
  - Log lines may or may not be enriched by OTel SDK
  - Log lines may not have all the necessary information eg trace_id
- Responsibilities of the agent
  - To manipulate the log lines to conform to OTel SDK with `filelog` receiver
  - Perform application specific transformation on the log lines (or other signals) with `transform` processor
  - Acts as a buffer for signals, prevent data from overwhelming the gateway collector

© Copyright National University of Singapore

# Logging Pipeline with Agent




© Copyright National University of Singapore

# `filelog` Receiver

- Receiver for ingesting logs, in text, transforming them into OTel log record format
- Features
  - Option to specify log file patterns to include or exclude
  - Checkpoints log files to ensure that when the collector restarts, parsing will resume from the previous location
  - Support log file rotation and log file compression
    - Eg `app.log.1`, `app.log.2.gz`
  - Aggregating over multiple lines eg Java stack trace
- Rich parsing support include JSON, regex, CSV, key value, syslog
  - Parsers can automatically extract information like timestamp, severity, and trace from log lines

© Copyright National University of Singapore

# Otel Collector Agent Configuration

```
receivers:
  filelog:
    include:
    - /var/logs/*.log
    operators:
    ...

processors:
  batch:

exporters:
  otlp:
    endpoint: otel-gateway:4317
    tls:
      insecure: true
```

Location of log files. Assume Winston writes logs here

Scrape and parses local log files

Process the logs, extract information

```
service:
  pipelines:
    logs:
      receivers: [ filelog ]
      processors: [ batch ]
      exporters: [ otlp ]
```

© Copyright National University of Singapore

# Processing Logs

- Create pipeline to parse, extract and transform log files
- Operators - pipeline steps
  - Parser to breakdown the log line to key/value pair, mandatory
    - Eg JSON, regex, csv, key/value,
  - Transformers to manipulate key/value pair e.g. move, copy, add, remove
- Common operations
  - Extracting the information like timestamp, error level and to use it to set OTel log records
  - Parsing data from unstructured logs
  - Redacting sensitive information like email, password, etc.

© Copyright National University of Singapore

# Example - Operators example

```
{"service.name":"books","service.version":"1.0.0","status":500,"level":"error","trac
e_id":"cf0...","span_id":"d14...","trace_flags":"01"}
```

**body** is the log line

Log fields

```
operators:
- type: json_parser
  parse_from: body
  parse_to: attributes
  severity:
    parse_from: attributes.level
- type: add
  field: attributes.env
  value: '${NODE_ENV}'
```

Parses the body to **attributes**. Sets the severity from entry with the value from **arributes.level**

Add a new field in attributes entry call env with the value from the environment variable **NODE_ENV**

Starts with a parser to extract fields from the log line.
Use operations like add, remove, move, extracted fields.

Value expression https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/pkg/stanza/docs/types/expression.md

© Copyright National University of Singapore

# Operators

- Operators define a series of steps to transform the log record
- Output result of one step becomes the input to the next
  - Output from the final step is the desired log record structure
- Every operator step consist of a **`type`**
  - Defines the action to be performed eg. **`parse_json`**
    - See list of available types https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/pkg/stanza/docs/operators/README.md#what-is-an-operator
  - **`type`** attribute define how the action should be perform
    - Eg. Parse a log line with key/value parser, and extract the timestamp from the key time
    - Eg. Move the attribute name status to resource attribute

© Copyright National University of Singapore

# Example - Parsers

```
2026-03-02 INFO User successfully logged into the system
```

Parse unstructured text

```
operators:
- type: regex_parser
  regex: '^(?P<time>\d{4}-\d{2}-\d{2}) (?P<level>\w+) (?P<message>.*)$'
  timestamp:
    parse_from: attributes.time
    layout: '%Y-%m-%d'
  severity:
    parse_from: attributes.level
```

Update the timestamp and severity with the parsed values from the log line

Extract pattern into log attributes

Parse JSON log line

```
operators:
- type: json_parser
  timestamp:
    parse_from: attributes.timestamp
    layout: '%Y-%m-%dT%H:%M:%S.%LZ'
```

Parse/extract from the log line and add to log attributes

© Copyright National University of Singapore

# Example - Parsers

```
service.name=books service.version=1.0.0 status=500 level=error trace_id=cf0...
span_id=d14... trace_flags=01
```

```
operators:
- type: key_value_parser
  pair_delimeter: ' '
  delimeter: '='
  ...
```

© Copyright National University of Singapore

# Example - Attribute Transformation

```
operators:
  - type: add
    field: resource["service.name"]
    value: "my-service"
  - type: move
    from: attributes.msg
    to: body
  - type: remove
    field: attributes.user_id
  - type: copy
    from: attributes.trace_id
    to: resource["trace_id"]
```

Add the literal `my-service` to the log resource

Move the `msg` field from log attribute to the log body

Remove `user_id` from the log attribute

Copy `trace_id` from log attribute to resource attribute

List of actions.
Executed sequentially
Each action has its own set of attributes

© Copyright National University of Singapore

# Example - Conditions

Use the JSON parse if the log line is enclosed in { }

```
- type: json_parser
  timestamp:
    parse_from: attributes.time
    layout: '%Y-%m-%d'
  severity:
    parse_from: attributes.level
  if: 'body matches "^\\s*\\{.*\\}\\s*$"'
```

Redact email address from the log in production environment

```
- type: regex_replace
  field: body
  pattern: '\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'
  replacement: '[EMAIL_REDACTED]'
  if: 'attributes.environment == "production"'
```

See expression language https://github.com/expr-lang/expr/blob/master/docs/language-definition.md

© Copyright National University of Singapore

# Log Fields

- Operator operations can access the log fields with the following prefix
  - **`timestamp`** - log entry's timestamp
  - **`resource`** - log resource, contains standard attributes like severity, span_id, service.name etc.
  - **`attributes`** - log attributes, populated by the logger. Contains information specific to the log
  - **`body`** - the log message

- Field expression is used in **`field`**, **`value`**, **`from`**, **`to`**

```
- type: copy
  from: attributes.user_id
  to: resource["user.id"]

- type: add
  field: resource["service.name"]
  value: "my-service"
```

© Copyright National University of Singapore

# Workshop 2
# Task 2

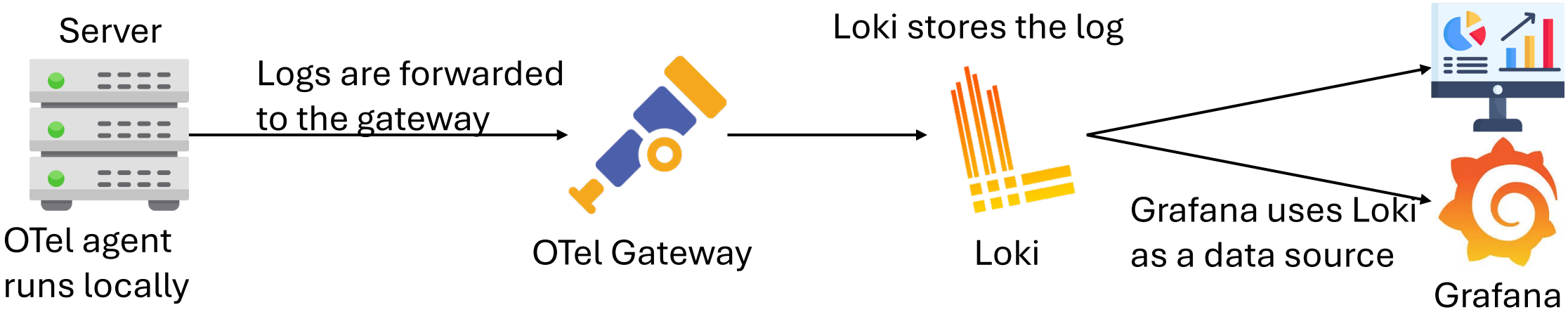
© Copyright National University of Singapore

# Forwarding Logs

Directly to Gateway

© Copyright National University of Singapore

# Forwarding Logs to Gateway




- Logs are written directly by the application logger to the gateway
  - Protocol can be HTTP or gRPC
- Use case for this will be containers running on platform as a service like Railway, Rendr, etc.
  - No local storage for logs
  - Lightweight but incur more network traffic
- Unified observability pipeline
  - Integration with other signals using connectors

© Copyright National University of Singapore

# OTel Collector Gateway Configuration

```yaml
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:

exporters:
  otlphttp/loki:
    endpoint: http://loki:3100/otlp
    tls:
      insecure: true
```

```yaml
service:
  pipelines:
    logs:
      receivers: [ otlp ]
      processors: [ batch ]
      exporters: [ otlp/loki ]
```

Pipeline for processing logs

otlphttp exporter exports logs, traces and metrics over HTTP using the OTLP format

Loki OTLP endpoint for log ingesting

© Copyright National University of Singapore

# OTel Logging SDK

- Provides a standardized APIs for collecting, processing and exporting logs from application

```
const { OTLPLogExporter } = require('@opentelemetry/exporter-logs-otlp-grpc')

const otlpLogExporter = new OTLPLogExporter({
  url: 'http://otlp-gateway:4317'
})

const sdk = new NodeSDK({
  logRecordProcessors: [
    new BatchLogRecordProcessor(otlpLogExporter)
  ],
  ...
})
```

Create an instance of **OTLPLogExporter** specifying the gRPC port of the OTLP gateway

Configure batching and add the log exporter instance to NodeSDK

© Copyright National University of Singapore

# Example - Writing a Log Line

Get an instance of the OTel logger

```
const { logs } = require('@opentelemetry/api-logs')
const metadata = require('package.json')

const logger = logs.getLogger(metadata.name, metadata.version)
```

Specify the log level and summary text

Log payload, can be text or structured data like JSON

```
try {
  ...
} catch (error) {
  logger.emit({
    severityNumber: SeverityNumber.ERROR,
    severityText: 'HTTP request error',
    body: error,
    attributes: { ... }
  })
}
```

Key/value pairs to be added to the logs attribute

⚠️ We can either use Winston or OpenTelemetry logger for logging

© Copyright National University of Singapore

# Example - Log Record

```
logger.info("level: info, searchBooksByTitle: cubicle=1, ... ")
```

```
ObservedTimestamp: 2025-10-06 03:32:31.866605962 +0000 UTC
Timestamp: 1970-01-01 00:00:00 +0000 UTC
SeverityText: info
SeverityNumber: Info(9)
Body: Str({"level":"info","message":"searchBooksByTitle:
cubicle=1","service.name":"books","service.version":"1.0.0","span_id":"66ceeaab7f03e902","t
imestamp":"2025-10-
06T03:32:31.828Z","trace_flags":"01","trace_id":"b40ddcc1e3a995c71a34629b9ed162ce"})
Attributes:
   trace_flags: Str(01)
   log.file.name: Str(books.log)
   span_id: Str(66ceeaab7f03e902)
   trace_id: Str(b40ddcc1e3a995c71a34629b9ed162ce)
   level: Str(info)
   message: Str(searchBooksByTitle: cubicle=1)
   service.name: Str(books)
   service.version: Str(1.0.0)
   timestamp: Str(2025-10-06T03:32:31.828Z)
```

Use these to populate the missing resource fields

© Copyright National University of Singapore

# Log Record

- Important to set the correct severity number
  - Can be use as a filter
- **`body`** is for capturing the log details
  - Can be structured log format like logfmt, JSON, syslog, etc
  - Or just unstructured text eg. User fred failed authentication
- **`attributes`** are attached to the logs
  - Additional metadata not auto injected by auto instrumentation
  - Note that these attributes are not resource attributes

© Copyright National University of Singapore

# Transform Processor

- Process to transform telemetry signals as it flows through the collector
  - Like `filelog` operator but can be applied to all signals

| Receiver | Transformer (Processor) | Exporters |
|---|---|---|

- Use cases
  - Add, modify, delete attributes
  - Enrich signals with contextual information
  - Redact sensitive data
  - Normalize inconsistent fields
  - Convert data types or formats
  - Filter and sample based on specific conditions

© Copyright National University of Singapore

# Example - transform Processor

Use the transform processor

All following statements will be applied to log signal

The following statements operate on the **log** context

```
processors:
  transform:
    log_statements:
    - context: log
      statements:
      - set(resource.attributes["environment"], "prod")
      - replace_pattern(body, "\\b\\d{4}[\\s-]?\\d{4}[\\s-]?\\d{4}[\\s-]?\\d{4}\\b", "XXXX-XXXX-XXXX-XXXX")
      - delete_key(attributes, "environment")
```

Statements are executed sequentially

© Copyright National University of Singapore

# OTTL - OpenTelemetry Transformation Language

- Language to manipulate telemetry signal
- Contains the following
  - Statements specify the transformation to be applied to the signal
  - Context specifies the domain (eg logs, traces, or metrics) in which the sentence will be operation on
  - Functions are operations used by statements to perform the transformation operation
- Available context, not exhaustive
  - **`resource`**
  - **`log`**
  - **`span`**
  - **`metric`**
- Each type of signal has their own specific context
  - Eg. log - **`body`**, **`severity_text`**, **`severity_number`**
  - See https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/pkg/ottl/contexts/ottllog/README.md
- Context are used to specify what attributes to update in the OTel recoreds
- Introduction to OTTL
  - https://bindplane.com/blog/what-is-the-opentelemetry-transform-language-ottl

© Copyright National University of Singapore

# Context

- **`resource`** - https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/pkg/ottl/contexts/ottlresource/README.md
  - OTLP resource
- **`log`** - https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/pkg/ottl/contexts/ottllog/README.md
- **`span`** - https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/pkg/ottl/contexts/ottlspan/README.md
- **`metric`** - https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/pkg/ottl/contexts/ottlmetric/README.md

© Copyright National University of Singapore

# Example - OTTL Functions

- Sets an attribute in **`log.attributes`**

```
set(attributes["environment"], "prod")
```

- Delete an attribute from **`log.attributes`**

```
delete_key(attributes, "environment")
```

- Replace strings from the **`body`** (log line)

```
replace_pattern(body, "\\d{3}-\\d{2}-\\d{4}")
```

- Conditional functions

```
set(resource.attributes["attention"], true) where severity_number > 15
```

- See https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/pkg/ottl/LANGUAGE.md

© Copyright National University of Singapore

# Example - Conditional Statements

Add latency attribute to a span record

```
processors:
  transform/latency_classification:
    trace_statements:
    - context: span
      statements:
      - set(attributes["latency"], "critical") where duration >= 5000000000
      - set(attributes["latency"], "high") where duration >= 3000000000 and
duration < 5000000000
      - set(attributes["latency"], "medium") where duration >= 1000000000 and
duration < 3000000000
      - set(attributes["latency"], "critical") where duration < 1000000000

...
services:
  pipelines:
    traces:
      receivers: [ otlp ]
      processors: [ transform/latency_classification ]
      exporters: [ otlp/tempo ]
```

© Copyright National University of Singapore

# Multi Stage Transformation

- Create specific transform performing a single type of transformation
  - Eg redacting, classifying, normalizing, etc
- Chain transform processors when configuring pipeline

Chaining 2 transform in a pipeline executed sequentially

```
service:
  pipelines:
    logs:
      receivers: [otlp]
      processors: [transform/redact, transform/enrich, batch]
      exporters: [otlp/loki]
```

© Copyright National University of Singapore

# Transform Processor vs Filelog Operators

- Both performs similar type of operations
- Filelog specifically for ingesting log files 1
  - Transformation occurs in receivers, during file ingesting
- Transform processor 2
  - Transformation occurs in processors, after ingestion
  - More generic, can transform traces, logs and metrics

| Pipeline | | |
|---|---|---|
| Receiver | Processor | Exporter |
| 1 `filelog` | 2 `transform` | `otlphttp` |
| `otlp` | | `otlp/tempo` |

© Copyright National University of Singapore

# Indexes

© Copyright National University of Singapore

# Index and Structured Metadata

- Labels are indexed metadata
  - Typically low cardinality eg **`service.name`**, **`service.version`**, **`level`**
  - Use for stream grouping
  - Labels cannot have `.` in their name; otherwise will be ignored by Loki
    - Eg. **`service_name`**, not **`service.name`**
- Structured metadata are non-indexed metadata
  - Use for filtering log lines, after grouping
- Loki looks for names in resource entry for indexing
  - Specify by **`default_resource_attributes_as_index_labels`** in Loki configuration file
  - Eg. **`service.name`**, **`k8.container.name`**, **`service.namespace`**
    - Changes the `.` to `_`

© Copyright National University of Singapore

# Creating Log Indexes

OTel collector agent

```
receivers:
  filelog:
    ...
  operators:
  - type: json_parser
    severity:
      parse_from: attributes.level
  - type: add
    field: resource["service.name"]
    value: attributes["service.name"
 - type: add
    field: resource["service.version"]
    value: attributes["service.version"]
 - type: add
    field: resource["level"]
    value: attributes["level"]
```

Loki

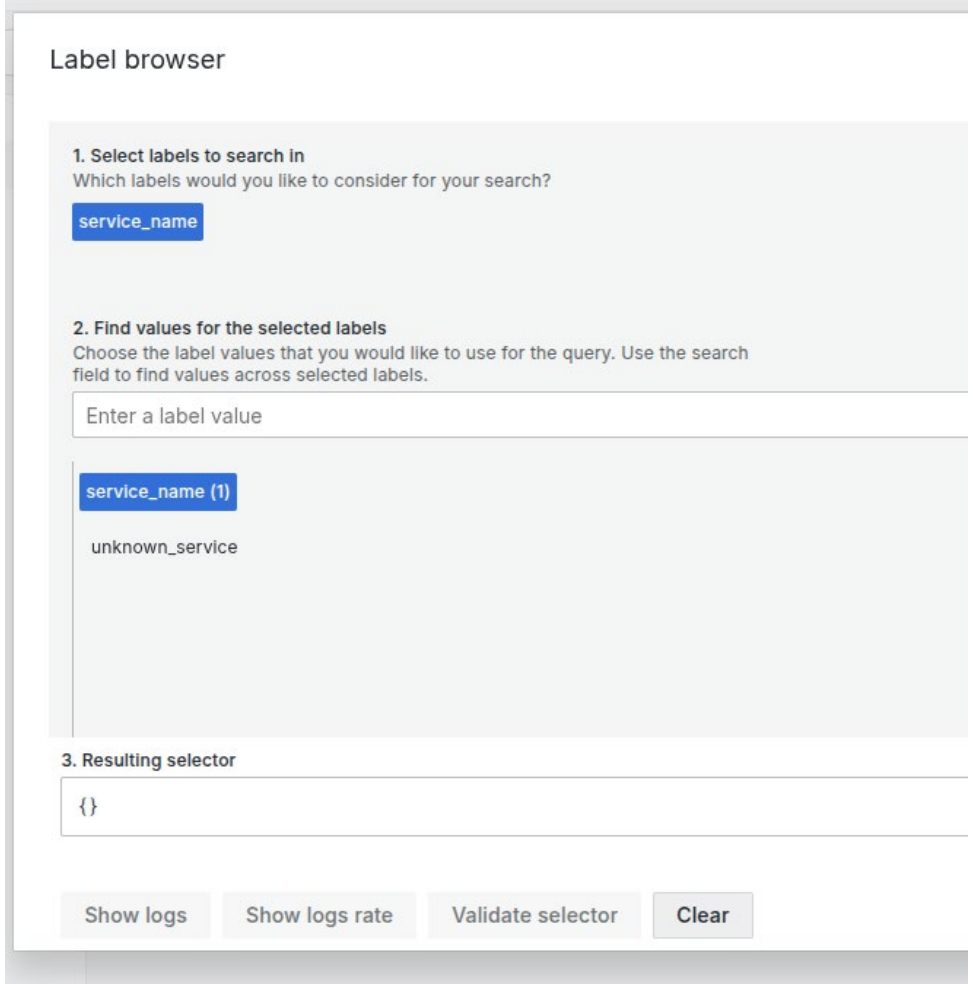
```
limits_config:
  allow_structured_metadata: true

distributor:
  otlp_config:
    default_resource_attributes_as_index_labels:
    - service.name
    - service.version
    - level
```

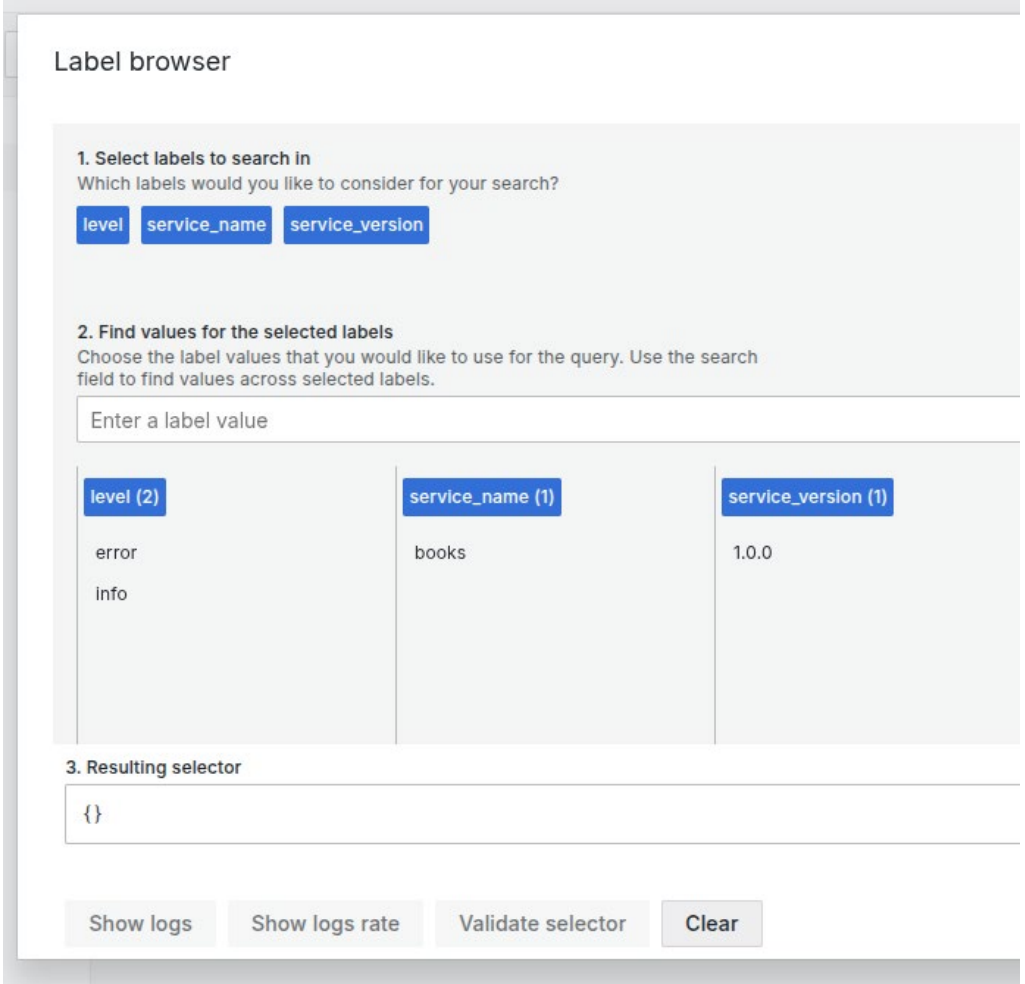
Loki can only create index for attributes in resource

© Copyright National University of Singapore

# Index




No appropriate labels found from log entry




Indexed 3 labels from log entry

© Copyright National University of Singapore

# Structured Metadata

2025-10-09 09:36:37.013 GET

I - index
S - structured metadata

Fields

| | |
|---|---|
| detected_level | |
| flags | |
| host_arch | amd64 |
| host_id | b93676df0cfa42289ae466b615b58c1b |
| host_name | batcomputer |
| observed_timestamp | 1759973797013000000 |
| process_command | /home/cmlee/src/soda/cno/solutions/day02/books/src/main.js |
| process_command_args | ["/usr/bin/node","--require","./src/instrument.js","/home/cmlee/src/soda/cno/solutions/day02/books/src/main.js"] |
| process_executable_name | node |
| process_executable_path | /usr/bin/node |
| process_owner | cmlee |
| process_pid | 54873 |
| process_runtime_description | Node.js |
| process_runtime_name | nodejs |
| process_runtime_version | 24.0.1 |
| scope_name | books |
| scope_version | 1.0.0 |
| service_name | books |
| service_name_extracted | books |
| service_version | 1.0.0 |
| service_version_extracted | 1.0.0 |

© Copyright National University of Singapore

# Workshop 2
# Task 3

© Copyright National University of Singapore

# Querying Logs

With LogQL

© Copyright National University of Singapore

# LogQL

- Loki's query language
  - Like SQL but for logs
  - Works with labels eg **`service_name=nginx`**
  - Syntax is inspired by Prometheus and is very similar to TraceQL
- Uses stream-based querying
  - View logs as a continuous stream, with labels as identifiers
  - Create specific log streams by grouping/filtering its labels
    - Logs with identical labels are included in the stream
  - Unlike full text search
- Optionally extract metrics from log streams
  - For visualization and altering
- Types of queries
  - log queries - search through raw log lines
  - metric queries - aggregate log lines into time-series metrics e.g. error rate
    - Derive metrics from logs if information is available in logs

© Copyright National University of Singapore

# LogQL Query Structure

**metric (stream selector → pipeline)**

```
count_over_time (
    {service_name="nginx"} | json | status >= 500 [5m]
)
```

- Log query consist of 2 parts
  - Log stream selector
    - What are the logs you are interested in?
    - Use labels
  - Log pipeline
    - What do you want to do with these logs?
    - Parse, filter, transform
- Optionally convert the query into a metric

- Pipelines are constructed with
  - Line filters
    - `|=, !=, |~, !~`
    - Eg. `|= "error"`
  - Parsers
    - `| json, | logfmt, | regex`
    - Eg. `| json`
  - Label filters
    - `| http.status_code >= 500`

© Copyright National University of Singapore

# Examples - Log Stream

```
{"service.name":"books","service.version":"1.0.0","status":500,"level":"error","trace_id":"cf0...","span_id":"d14...","trace_flags":"01", ...}
```

- Find all **`books`** service name logs
  - Loki converts label names with dots (`.`) to underscore (_)
  - Eg. **`service.name`** to **`service_name`**

```
{ service_name = "books" }
```

- Find all version 1.x of the **`books`** services

```
{ service_name = "books", service_version =~ "1\\..+$" }
```

© Copyright National University of Singapore

# Examples - Pipeline - 1

```
{"service.name":"books","service.version":"1.0.0","status":500,"level":"error","trace_id":"cf0...","span_id":"d14...","trace_flags":"01", ...}
```

- Filter lines with the literal **`error`**
  - Simple string match

```
{ service_name = "books" } |= "error"
```

- Look for all logs belonging to a trace
  - If logs are structured, use the appropriate parser to destructure the log line to their labels/attributes
  - Eg. use JSON parser, attributes are accessible after the parser

```
{ service_name = "books"} | json | trace_id = "cf0..."
```

  - Eg. match log body without parsing

```
{ service_name = "books"} |= "cf0..."
```

© Copyright National University of Singapore

# Examples - Pipeline - 2

{"service.name":"books","service.version":"1.0.0","status":500,"level":"error","trace_id":"cf0...","span_id":"d14...","trace_flags":"01", ...}

- Find all 5xx errors

```
{ service_name = "books"} | json | status >= 500
```

- Ignore all **/health** resource

```
{ service_name = "books" } | json | http.resource != "/healthz"
    | http.method =~ "(GET|POST)"
```

© Copyright National University of Singapore

# Examples - Pipeline - 3

```
10.0.0.50 - - [15/Jan/2024:10:31:15 +0000] "POST /api/submit" 404 567
```

- Parse and extract the IP address, method, URL and status code from an unstructured log line

```
{ service_name = "nginx" }
  | regex `(?P<ip>\d+\.\d+\.\d+\.d+)`
  | regex `"(?P<method>\w+) (?<url>\S+)"`
  | regex `(?P<status>\d{3})`
```

- Find all failed submission
  - Regex parsed labels are string

```
... | url = "/api/submit" | status =~ "2[0-9][0-9]"
```

© Copyright National University of Singapore

# Metric Queries

- LogQL queries can be used to construct time series data
  - Has to be enabled with **`metric_aggregation_enabled`** in Loki's configuration file
- Operation includes
  - Adding a range vector selector (time window) to the query
    - Eg **`[5m]`**, **`[$__range]`**
  - Applying one or more aggregation functions to the range vector
    - Range aggregation - **`rate`**, **`count_over_time`**, **`sum_over_time`**
      - Eg. **`count_over_time(...[5m])`**
      - See https://grafana.com/docs/loki/latest/query/metric_queries/#log-range-aggregations
    - Aggregation function - **`sum`**, **`min`**, **`max`**, **`avg`**
      - Eg. **`sum(rate(...[5m]))`**
      - See https://grafana.com/docs/loki/latest/query/metric_queries/#built-in-aggregation-operators

© Copyright National University of Singapore

# Example - Metric Queries

- Count the number of submission errors over the last 5 minutes

```
count_over_time( { service_name = "books" } | json
  | severity_number = 17 | url_path = "/submit" [5m])
```

- Rate of submission request over user selected time window

```
rate( { service_name = "books" } | json
  | http_request_method = "POST" | url_path = "/submit" [$__range])
```

- Calculate the number of bytes transferred over 5 minutes
  - Assuming Nginx access log format

```
count_over_time(
  { service_name = "nginx" }
    | regex `^(?P<_>.+ "(?P<_>.+)" (?P<_>\d{3}) (?P<xfer_bytes>\d+) .+$`
    | unwrap xfer_bytes [5m]
)
```

© Copyright National University of Singapore

# Working with Time Series

- Range queries can potentially return my time series
  - with the **`_over_time`** functions

```
count_over_time(
  {service_name = "books" } | logfmt | severity_number = 17 [5m]
)
```

  - Log lines with different label values are considered a time series

```
a0=x a1=y a2=z
a0=a a1=y a2=z
a0=p a1=q a2=r
a0=x a1=y a2=z
```

3 time series

- Common to group the range vectors by specific labels values to reduce the number of time series data

```
count(
  count_over_time(
    {service_name = "books" } | logfmt | severity_number = 17 [5m]
  )
) by (service_name, service_version)
```

© Copyright National University of Singapore

# Workshop 2
# Task 4

© Copyright National University of Singapore

# Trace to Log Correlation

© Copyright National University of Singapore

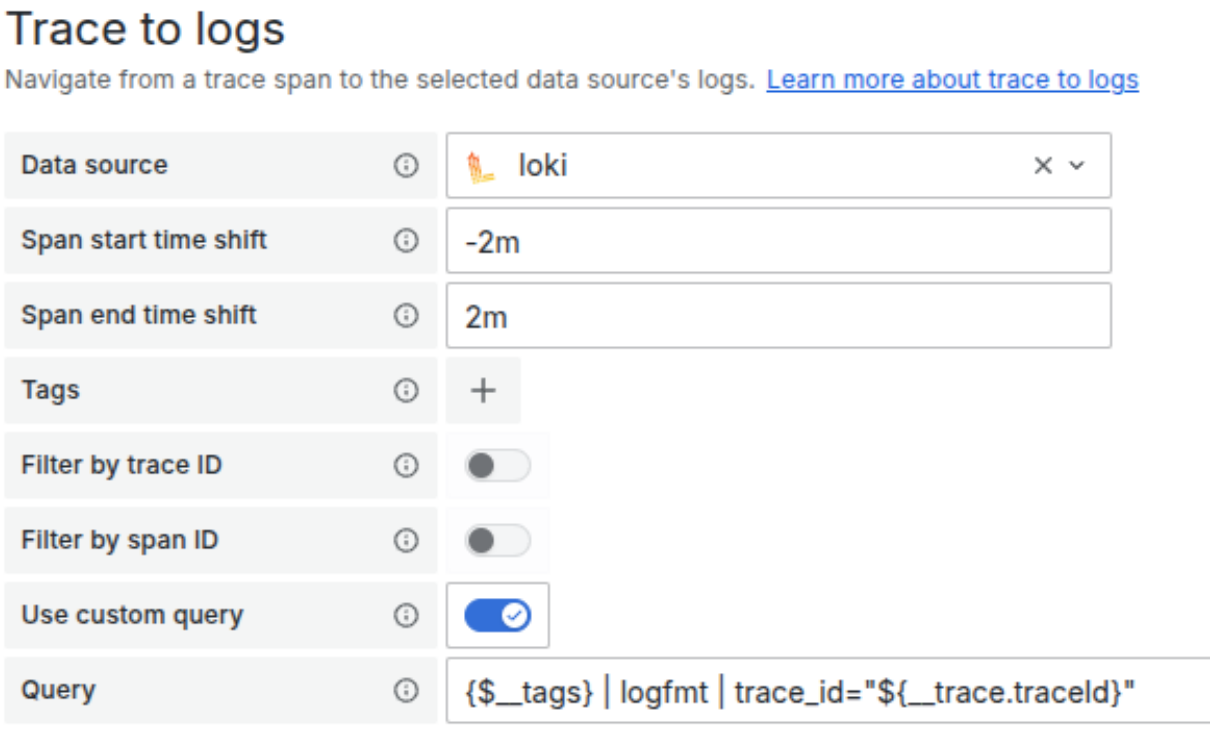
# Trace to Log Correlation

- Different perspectives
  - Log - what went wrong eg. exception stack, error logs, events
  - Trace - where in the system, the problem occurs eg. which service, in which component
- Helps to pinpoint the issue
  - Eg. when a specific trace exhibits latency, jump to the log of that specific trace
- Important that logs contain trace id and span id
  - Automatically added to the log record by auto instrumentation
  - Manually patch log record with trace and span id if the logs are scaped from log files
  - Use other attributes as identities to correlate trace and span id is not available

© Copyright National University of Singapore

# Configuration

- Add Loki to Tempo data source
  - Need to add Loki data source first
- Configure how to correlate a trace to log
  - How to match trace and span id from traces to logs
    - Filter by trace and/or span ID - auto generated by Grafana
    - Use custom query - works better
      - The LogQL query to execute to get logs corresponding to the trace
- Time shift
  - The duration to look for the log for correlation




© Copyright National University of Singapore

# Custom Query Variables

- Custom query variables allows you to access the context of the source signal
  - Correlating from trace to log, then trace is the source signal
- Access by wrapping these variables in `${}`
- `__tags` is a place holder to capture the filters from TraceQL
  - Capture labels are dynamically converted to LogQL stream selector
- Common query variables
  - `__span.spanId`
  - `__trace.traceId`
- See https://grafana.com/docs/grafana/latest/datasources/tempo/configure-tempo-data-source/#custom-query-variables

© Copyright National University of Singapore

# Example - Trace to Log - 1

Home › Explore › tempo | tempo

Search... ctrl+k | Sign in

Query history | Share

× Close | Add

Queries
Table

A (tempo)

Query type: Search | TraceQL | Service Graph | Import trace

Build complex queries using TraceQL to select a list of traces. Documentation

{status=ok}

Search Options | Limit: 20 | Spans Limit: 3 | Table Format: Traces | Streaming: Disabled | Metrics Options | Step: auto | Type: Range

+ Add query | Query inspector

Table - Traces

| Trace ID | Start time | Service | Name |
|---|---|---|---|
| 273cf18c6c1c29... | 2025-10-13 10:25:16. | books | POST /api/ |
| f34b226b1c20d... | 2025-10-13 10:25:02. | books | GET /api/se |
| f2889cec2e3c9... | 2025-10-13 10:24:56. | books | GET /api/b |
| 993226ca1d6bf... | 2025-10-13 10:24:56. | books | GET /api/b |
| d3b47b973629f... | 2025-10-13 10:24:52. | books | GET /api/b |
| d6adc7ba6300... | 2025-10-13 10:24:51. | books | POST /api/ |
| 6f58b3ef6ba8b... | 2025-10-13 10:24:49. | books | GET /api/b |

| Span ID | Start time | status |
|---|---|---|
| ba2a06f92118f5a7 | 2025-10-13 10:25:16.461 | ok |
| 72fa82925afc102a | 2025-10-13 10:25:16.460 | ok |

× Close | Add to dashboard

Queries
Traces

Query type: Search | TraceQL | Service Graph | Import trace

Build complex queries using TraceQL to select a list of traces. Documentation

273cf18c6c1c297b6d9f224d54d6e8aa

Search Options | Limit: 20 | Spans Limit: 3 | Table Format: Traces | Streaming: Disabled | Metrics Options | Step: auto | Type: Range

+ Add query | Query inspector

Trace

books: POST /api/book 13.69ms
2025-10-13 10:25:16.460 /api/book

Give feedback | Trace ID | Export

Span Filters | 7 spans | Prev | Next

0µs | 3.42ms | 6.85ms | 10.27ms | 13.69ms

Service & O... | 0µs | 3.42ms | 6.85ms | 10.27ms | 13.69ms

- books POST /api/book (13.69ms)
  - router - /api (9.87µs)
  - middleware - <anonymous> (17
    - POST:/api/book (13.36ms)
      - insertBook (12.52ms)
        - (12.16ms)
  - request handler - /api/book (13

© Copyright National University of Singapore

# Example - Trace to Log - 2

Home › Explore › tempo | loki

Search... ctrl+k | Sign in

Query history | Share

Close | Add

Queries
Table

A (tempo)

Query type: Search | TraceQL | Service Graph | Import trace

Build complex queries using TraceQL to select a list of traces. Documentation

```
{status=ok}
```

Search Options | Limit: 20 | Spans Limit: 3 | Table Format: Traces | Streaming: Disabled | Metrics Options | Step: auto | Type: Range

+ Add query | Query inspector

Table - Traces

| Trace ID | Start time | Service | Name |
|---|---|---|---|
| 273cf18c6c1c29... | 2025-10-13 10:25:16. | books | POST /api/ |
| f34b226b1c20d... | 2025-10-13 10:25:02. | books | GET /api/se |
| f2889cec2e3c9... | 2025-10-13 10:24:56. | books | GET /api/b |
| 993226ca1d6bf... | 2025-10-13 10:24:56. | books | GET /api/b |
| d3b47b973629f... | 2025-10-13 10:24:52. | books | GET /api/b |
| d6adc7ba6300... | 2025-10-13 10:24:51. | books | POST /api/ |
| 6f58b3ef6ba8b... | 2025-10-13 10:24:49. | books | GET /api/b |

| Span ID | Start time | status |
|---|---|---|
| ba2a06f92118f5a7 | 2025-10-13 10:25:16.461 | ok |
| 72fa82925afc102a | 2025-10-13 10:25:16.460 | ok |

Add to dashboard

Queries
Logs

A (loki)

Kick start your query | Label browser | Explain query | Builder | Code

```
{service_name="books"} | logfmt |
trace_id="273cf18c6c1c297b6d9f224d54d6e8aa"
```

1 `{service_name="books"}`
Fetch all log lines matching label filters.

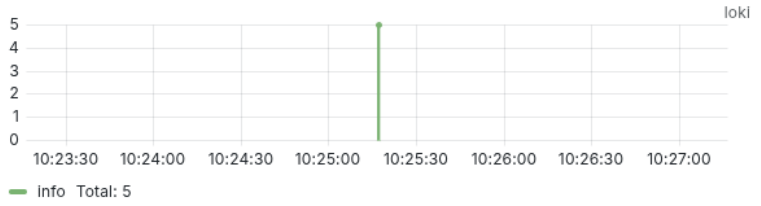
2 `<expr> | logfmt`
This will extract all keys and values from a logfmt formatted log line as labels. The extracted labels can be used in label filter expressions and used as values for a range aggregation via the unwrap operation.

3 `<expr> | trace_id = 273cf18c6c1c297b6d9f224d54d6e8aa`
Label expression filter allows filtering using original and extracted labels.

Options | Type: Range | Line limit: 1000 | Direction: Backward

+ Add query | Query inspector

Logs volume




Logs | Logs | Table

5 lines displayed Total bytes processed: 172 kB

© Copyright National University of Singapore

# Example - Custom Query

**TraceQL**

```
{ resource.service.name="books" && status=ok }
```

Trace ID from the result is substituted

```
{$__tags} | logfmt | trace_id="${__trace.traceId}"
```

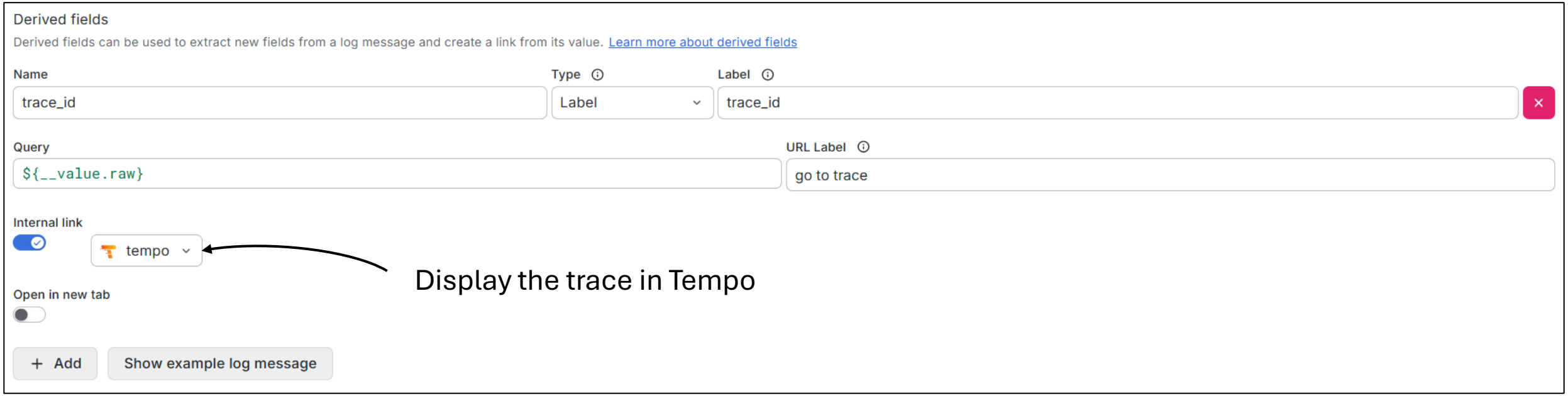
**LogQL**

```
{ service_name="books" } | logfmt | trace_id="<trace_id>"
```

© Copyright National University of Singapore

# Log to Trace Correlation

- Loki derived fields are used to link logs to traces
- By extracting fields from either the log line or from structured metadata
  - Label - structured metadata name
  - Log line - use regex, last option
- Extracted fields are stored in **`${__value.raw}`**




Display the trace in Tempo

© Copyright National University of Singapore

# Example - Log to Trace - 1

Home › Explore › loki

Search... ctrl+k

Sign in

Query history | Share

Outline | loki | Go queryless | Split | Add | Last 1 hour | Run query | Live

0

10:30 10:35 10:40 10:45 10:50 10:55 11:00 11:05 11:10 11:15 11:20 11:25

error Total: 2.51 K

Logs

Logs | Table

1000 lines shown – 38.79% (23min 16sec) of 1h Total bytes processed: 666 kB Common labels: db_collection_name=unknown flags=1 host_arch=amd64 +19

Search fields by name

Fields

| | |
|---|---|
| db_collection_name | 100% |
| db_collection_name_extracted | 39% |
| detected_level | 100% |
| flags | 100% |
| host_arch | 100% |
| host_id | 100% |
| host_name | 100% |
| observed_timestamp | 100% |
| process_command | 100% |
| process_command_args | 100% |
| process_executable_name | 100% |
| process_executable_path | 100% |

2026-01-14 11:28:18.667 ERROR Error injection triggered for /search: Service temporarily unavailable

2026-01-14 11:28:18.667 ERROR /api/search?q=tom Service temporarily unavailable

2026-01-14 11:28:16.959 ERROR searchBooksByTitle: tolkien=17

2026-01-14 11:28:16.953 ERROR searchBooksByTitle: search_term=tolkien, limit=20

2026-01-14 11:28:16.305 ERROR getBookByISBN13: 9780801435300=true

Search field names and values

Log line

Error injection triggered for /search: Service temporarily unavailable

Links

trace_id go to trace

Trace

Indexed label

db_collection_name unknown

service_name books

Expanded

Scroll to bottom

Newest logs first

Search logs

Deduplication

Filter levels

Display ms

Wrap JSON

Highlight text

Large font

# Example - Log to Trace - 2

Home › Explore › loki | tempo

Search... ctrl+k

Sign in

Query history

Share

Go queryless

Add

40
20
0
10:35 10:40 10:45 10:50 10:55 11:00 11:05 11:10 11:15 11:20 11:25 11:30

error Total: 2.45 K

Logs

Logs Table

1000 lines shown – 39.34% (23min 36sec) of 1h Total bytes processed: 724 kB

Common labels: db_collection_name=unknown flags=1 host_arch=amd64 +19

Search fields by name

Fields

| | |
|---|---|
| db_collection_name | 100% |
| db_collection_name_extracted | 39% |
| detected_level | 100% |
| flags | 100% |
| host_arch | 100% |
| host_id | 100% |
| host_name | 100% |
| observed_timestamp | 100% |
| process_command | 100% |
| process_command_args | 100% |

Expanded

Scroll to bottom

Newest logs first

Search logs

Deduplication

Filter levels

Display ms

Wrap JSON

Highlight text

tempo

Add to dashboard

A (tempo) 0cd949d3fc701d39ddb8785ac5afe5c8

Add query

Query inspector

Trace

books: GET /api/book/:isbn13

Feedback

Share

Trace ID 0cd949d3fc701d39ddb8785ac5afe5c8 Start time 2026-01-14 11:34:10.440 (5 minutes ago) Duration 1.3s

Services 1 Route /api/book/:isbn13

Span Filters

4 spans Prev Next

0µs 324.01ms 648.02ms 972.02ms 1.3s

Service & Operation

| | |
|---|---|
| books GET /api/book/:isbn13 (1.3s) | |
| router - /search (3.58µs) | 3.58µs |
| request handler - /api/book/:isbn13 (1.3s) | |
| getBookByISBN13 (203.34µs) | 203.34µs |

# Workshop 2
# Task 5, 6 and 7

© Copyright National University of Singapore

# End of Module

© Copyright National University of Singapore